

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Програмування. Частина 2.

Лабораторна робота №3

«Рядки в мові програмування Java (Python)»

Виконав:

студент гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив

Коренко Д.В.

Тема: «Рядки в мові програмування Java (Python)».

Мета: Ознайомлення з рядками та використання основних методів їх обробки в мові програмування Java (Python). Здобуття навичок у використанні рядків в мові програмування Java (Python).

Порядок роботи

Через те, що є можливість використовувати мову Python, для виконання лабораторних робіт з ООП, то я надалі буду його використовувати як основний засіб.

Моя залікова книжка 4106, значить:

$$C_3 = 4106 \bmod 3 = 2$$

$$C_{17} = 4106 \bmod 17 = 9$$

Згідно з таблицями варіантів, мій варіант:

C_3	Тип
0	StringBuilder
1	StringBuffer
2	String

C_{17}	Дія з рядком
...	...
8	Відсортувати слова заданого тексту за зростанням кількості в них заданої літери.
9	Задано текст та масив слів. Підрахувати у скількох реченнях зустрічається кожне слово масиву.
10	З кожного речення заданого тексту видалити підрядок найбільшої довжини, що починається та закінчується заданими літерами.
...	...

Код (Python):

```
import re
# бібліотека re -- регулярні вирази, задля розділення тексту на
речення, та для пошуку слів в цих реченнях

class LabSolver:

    def __init__(self, string: str, list_of_words: list):

        # ініціалізація атрибутів екземпляра

        self.list_of_words = list_of_words
        self.string = string

    def main_method(self) -> None:

        sentences = re.split(r'(?<=[.!?])\s+', self.string) #
Розбиваємо рядок на список, розділювачі: .!?
        self.list_of_sentences = [s.strip() for s in sentences if
s.strip()] # присвоюємо список речень без зайвих пробілів атрибута
екземпляра

        self.result = dict.fromkeys(self.list_of_words, 0) # На
основі списку слів, створюю словник, де ключ -- кожен елемент
списку, а значення всіх ключів за замовч. 0

        # allowed_chars -- набір символів, які ми вважаємо частиною
слова

        # Він містить латинські літери (A-Za-z), кириличні літери
(А-Яа-я) та специфічні для української мови символи (Ґґ, Єє, Іі, Ії)
        # а також апостроф (') і дефіс (-). Екранування дефіса (\\-)
потрібне, тому що дефіс має спеціальне значення всередині квадратних
дужок
        allowed_chars = "A-Za-zА-Яа-яҐґЄєІіІї'\\"

        # pattern_template -- шаблон регулярного виразу, який
перевіряє, чи слово з'являється як окремий блок
        # (?<![allowed_chars]) -- негативний lookbehind, що
гарантує: перед шуканим словом не йде символ із allowed_chars
        # {} - заповнювач, куди буде підставлено шукане слово (після
екранування)
        # (?![allowed_chars]) -- негативний lookahead, що гарантує:
після шуканого слова не йде символ із allowed_chars
        pattern_template = r'(?<[' + allowed_chars + r']){}(?![ ' +
allowed_chars + r'])'

        for sentence in self.list_of_sentences:

            # Перевіряємо кожне слово зі словника
            for word in self.list_of_words:

                # re.escape(word) -- екранує спеціальні символи в
слові, щоб вони трактувалися буквально
                # Підставляємо слово у шаблон, отримуючи повний
регулярний вираз, який шукає слово як окрему одиницю
                pattern = pattern_template.format(re.escape(word))

                # re.search шукає перше входження шаблону в реченні
                # Параметр flags=re.IGNORECASE робить пошук
нечутливим до регістру
```

```

        if re.search(pattern, sentence,
flags=re.IGNORECASE): self.result[word] += 1

    def output(self) -> None:

        for word in self.list_of_words: print(f"Слово
\"{word.title()}\": зустрічається в такій кількості речень:
{self.result[word]}")

# Як приклад, text -- вірш Ліни Костенко, "Людина нібито не літає...
А крила має"
text = """
А й правда, крилатим ґрунту не треба.
Землі немає, то буде небо.
Немає поля, то буде воля.
Немає пари, то будуть хмари.
В цьому, напевно, правда пташина...
А як же людина? А що ж людина?
Живе на землі. Сама не літає.
А крила має. А крила має!
Вони, ті крила, не з пуху-пір'я,
А з правди, чесноти і довір'я.
У кого - з вірності у коханні.
У кого - з вічного поривання.
У кого - з щирості до роботи.
У кого - з щедрості на турботи.
У кого - з пісні, або з надії,
Або з поезії, або з мрії.
Людина нібито не літає...
А крила має. А крила має!
"""

# Шукані слова
words = ["кого", "Крила", "Я", "з", "А", "має", "Gilbert"]

ex = LabSolver(text, words)
ex.main_method()
ex.output()

```

Результат виконання:

```

PS C:\Users\artem\OneDrive\Desktop\reps\university_roadmap\semestr2\oop\lab3> py .\lab.py
Слово "Кого" зустрічається в такій кількості речень: 5
Слово "Крила" зустрічається в такій кількості речень: 5
Слово "Я" зустрічається в такій кількості речень: 0
Слово "з" зустрічається в такій кількості речень: 6
Слово "А" зустрічається в такій кількості речень: 8
Слово "Має" зустрічається в такій кількості речень: 4
Слово "Gilbert" зустрічається в такій кількості речень: 0

```

Висновок:

У ході лабораторної роботи було вивчено особливості роботи з ООП та з рядками в мові програмування Python. Реалізовано програму, яка аналізує заданий текст і визначає кількість речень, у яких зустрічаються задані слова. Використання регулярних виразів значно полегшує та пришвидшує процес обробки тексту, дозволяючи чітко визначати межі слів та речень. Отримані навички будуть корисні при вирішенні різноманітних завдань обробки та аналізу текстових даних. Робота підтвердила зручність та ефективність Python у контексті роботи з рядками та регулярними виразами.